UNIVERSIDAD DE GUADALAJARA
Red Universitaria e Institución Benemérita de Jalisco

CENTRO UNIVERSITARIO DEL SUR

DIVISIÓN DE CIENCIAS EXACTAS, NATURALES Y TECNOLÓGICAS

DEPARTAMENTO DE CIENCIAS COMPUTACIONALES E INNOVACIÓN TECNOLÓGICA

INGENIERÍA EN TELEMÁTICA

UNIDAD DE APRENDIZAJE:

COMUNICACIÓN DIGITAL

PROFESOR DE LA UNIDAD DE APRENDIZAJE:
EMANUEL RODRIGO GUTIERREZ FIGUEROA

Documentación Final del Cansat "Heisenberg"

Paul Emmanuel Covarrubias Franco
Emmanuel Sánchez Sánchez
Hugo Fernando Ramos Chulin
Carlos Eduardo Villa Cuevas
David Aaron Camberos Baltazar

Ciudad Guzmán, Jalisco 19/Mayo/2025

índice

Graficas de Matlab.....3

Fotografías..... 4

Códigos de estación base, MATLAB y canstat.....8

Código estación base Arduino uno - 8

Código estación base MATLAB - 11

Código del cansat “Heisenberg” - 16

Conclusión final.....21

Graficas de Matlab

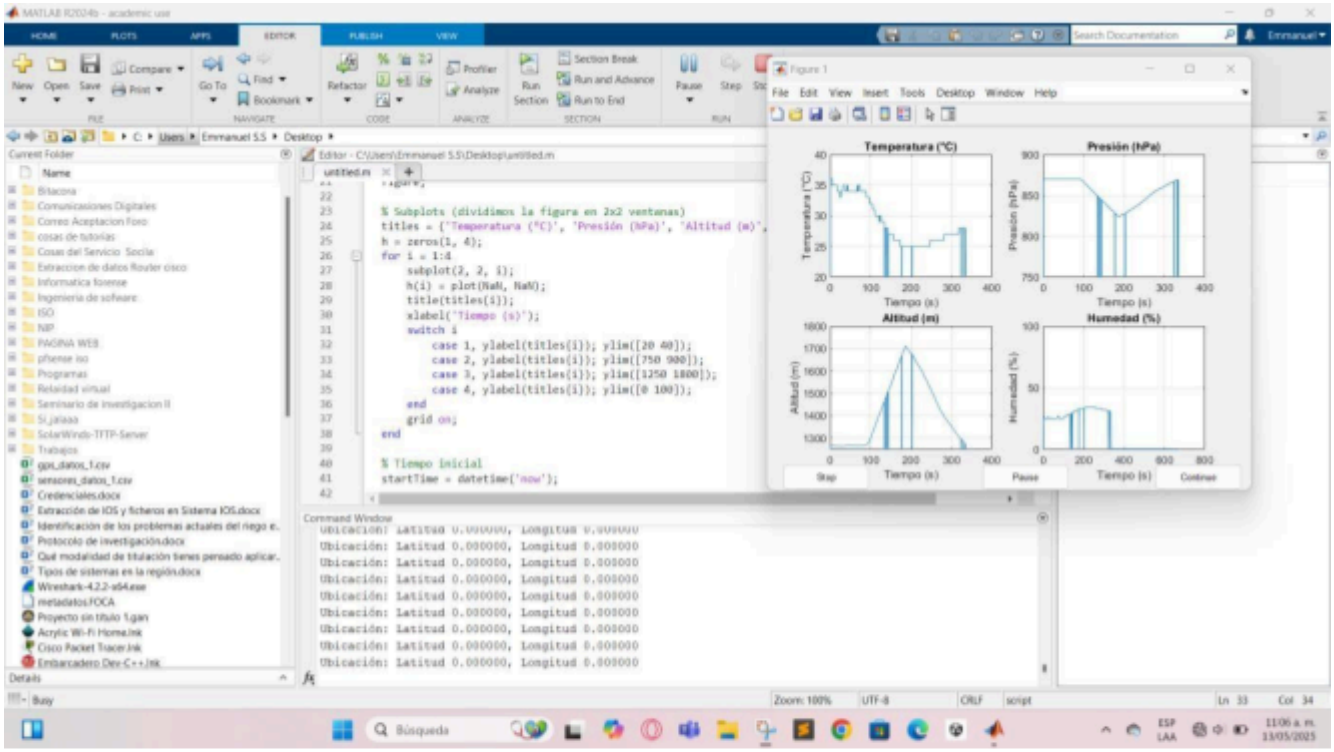


Imagen. 1.1 Captura de pantalla de las graficas de las cuatro variables.

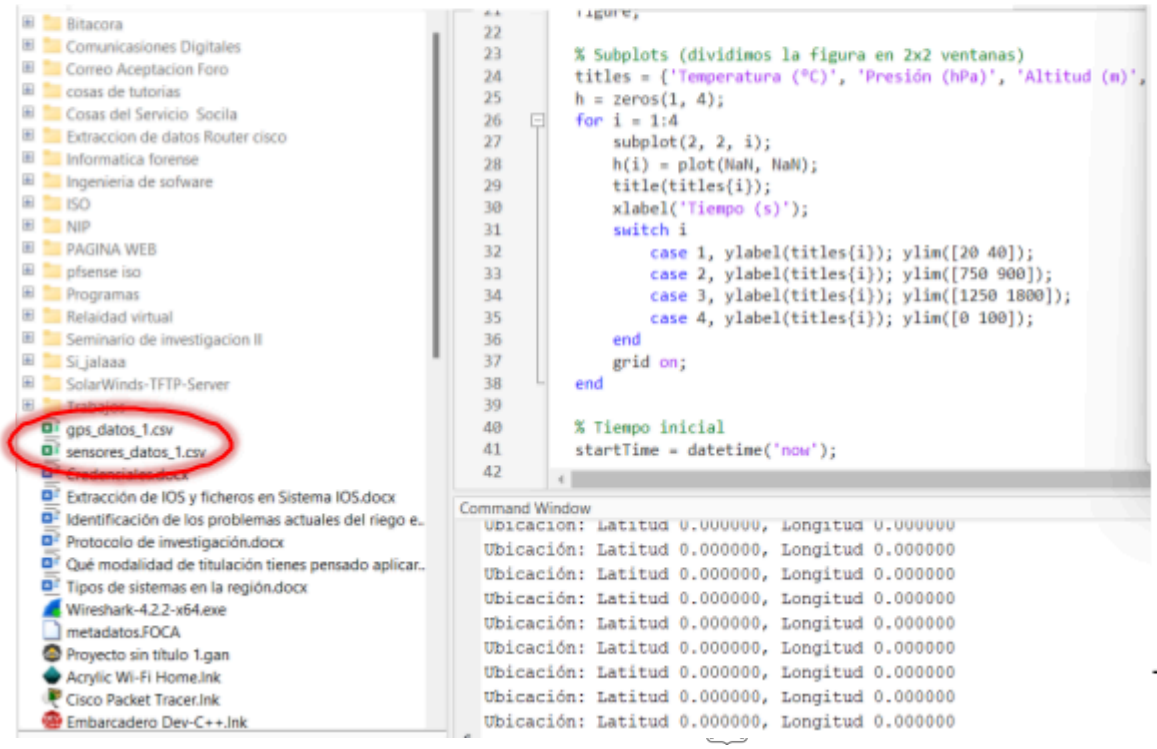


Imagen. 1.2 Captura de pantalla del código de MATLAB, a la izquierda se observan los documentos creados en Excel (ovalo rojo).

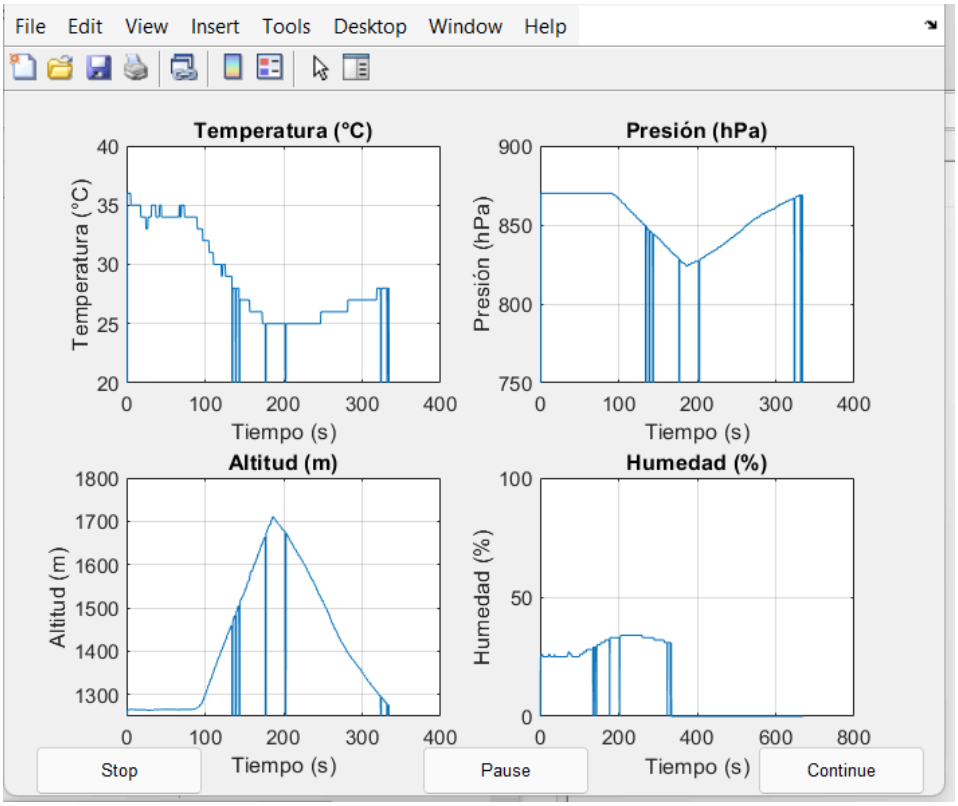


Imagen. 1.3 Captura de pantalla mas detallada de las variables de temperatura, presión, altitud y humedad con sus respectivas graficas.

Fotografías



Imagen. 1.5 En la izquierda se observa la estación base y al lado derecho el cansat “Heisenberg”.



Imagen. 1.6 Fotografía previa antes del vuelo, donde se muestra todo listo.



Imagen. 1.7 Cansat enganchado al dron preparado para subir.

nática.



Imagen. 1.8 Cansat subiendo.



Imagen. 1.9 Casat elevándose.



Imagen. 1.10 Casat ascendiendo.

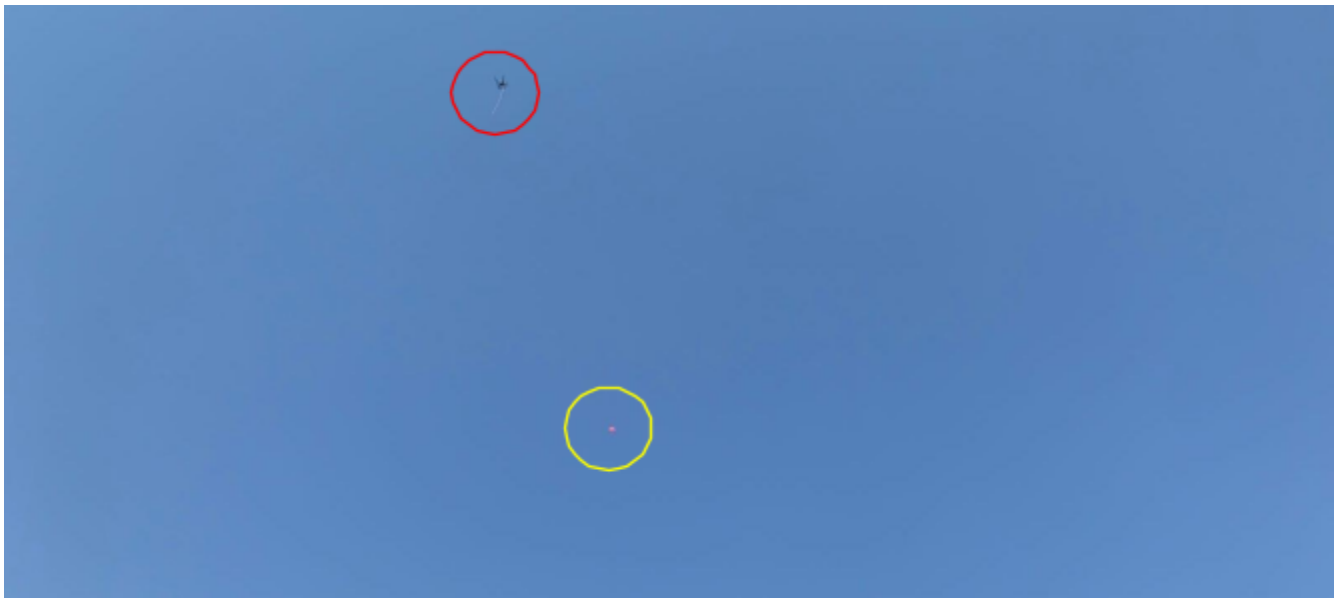


Imagen. 2.1 Casat liberado se nota un pequeño punto rosa (circulo amarillo) y el dron (círculo rojo).

Códigos de estación base, MATLAB y canstat

Código estación base Arduino uno

```
#include <SPI.h>
#include <RF24.h>

RF24 radio(8, 9); // Pines CE y CSN del nRF24L01

// Dirección del canal de comunicación (5 bytes)
const byte address[6] = "OCHO8";

// Estructura para almacenar los datos recibidos del transmisor
struct SensorData {
  float temperature; // Temperatura
  float pressure;    // Presión atmosférica
  float altitude;    // Altitud
  float humidity;    //
  Humedad float latitude;      //
  Latitud float longitude;
                     // Longitud
};

SensorData currentData;
unsigned long lastReceivedTime = 0;
const unsigned long timeoutDuration = 2000;

// Configuración del buzzer activo
const int buzzerPin = 7;
bool alarmaActiva = false;
unsigned long lastBuzzerToggle = 0;
bool buzzerState = false;
```

```
void setup() {
  Serial.begin(9600);

  // Inicializar radio nRF24L01
  if (!radio.begin()) {
    Serial.println("Fallo de inicialización de la radio");
    while (1);
  }

  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(4);
  radio.openReadingPipe(1, address);
  radio.startListening();

  // Inicializar variables
  currentData.temperature = 0;
  currentData.pressure = 0;
  currentData.altitude = 0;
  currentData.humidity = 0;
  currentData.latitude = 0;
  currentData.longitude = 0;

  // Configurar buzzer
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);
}

void loop() {
  // Verificar si hay datos disponibles
  if (radio.available()) {
    SensorData data;
```

```
  radio.read(&data,
  sizeof(SensorData)); currentData =
  data;
  lastReceivedTime = millis();
}

// Si no se reciben datos en el tiempo límite, resetear variables
if (millis() - lastReceivedTime > timeoutDuration) {
currentData.temperature = 0;
  currentData.pressure = 0;
  currentData.altitude = 0;
  currentData.humidity = 0;
  currentData.latitude = 0;
  currentData.longitude = 0;
}

// Enviar datos por serial en formato CSV
Serial.print(currentData.temperature);
Serial.print(","); Serial.print(currentData.pressure);
Serial.print(","); Serial.print(currentData.altitude);
Serial.print(","); Serial.print(currentData.humidity);
Serial.print(","); Serial.print(currentData.latitude, 6);
Serial.print(",");
Serial.println(currentData.longitude, 6);

// Control del buzzer por
temperatura if
(currentData.temperature >= 30) {
alarmaActiva = true;
}

if (alarmaActiva) {
  if (millis() - lastBuzzerToggle >= 500) {
```

```
lastBuzzerToggle = millis();
buzzerState = !buzzerState;
```

```
      digitalWrite(buzzerPin, buzzerState);
    }
  } else {
    digitalWrite(buzzerPin, LOW);
  }

  delay(500);
}
```

Código estación base MATLAB

```
clc;
close all;
clear all;

% Configuración del puerto serial
port = 'COM11'; % Cambia esto por el puerto serial de tu Arduino
baudRate = 9600;

% Crear el objeto serial
s = serial(port, 'BaudRate', baudRate, 'Terminator', 'LF');

% Intentar abrir el puerto serial con manejo de errores
try
    fopen(s);
    disp('Puerto serial abierto con éxito');
catch ME
    error('Error al abrir el puerto serial: %s', ME.message);
end

% Inicialización de variables para el gráfico
```

```
figure;

% Subplots (dividimos la figura en 2x2 ventanas)
titles = {'Temperatura (°C)', 'Presión (hPa)', 'Altitud (m)', 'Humedad (%)'};
h = zeros(1, 4);
for i = 1:4
    subplot(2, 2, i);
    h(i) = plot(NaN, NaN);
    title(titles{i});
    xlabel('Tiempo (s)');
    switch i
        case 1, ylabel(titles{i}); ylim([20 40]);
        case 2, ylabel(titles{i}); ylim([750 900]);
        case 3, ylabel(titles{i}); ylim([1250 1800]);
        case 4, ylabel(titles{i}); ylim([0 100]);
    end
    grid on;
end

% Tiempo inicial
startTime = datetime('now');

% Crear archivos CSV nuevos con nombres únicos
index = 1;
while true
    sensorFileName = sprintf('sensores_datos_%d.csv', index);
    gpsFileName = sprintf('gps_datos_%d.csv', index);
    if ~exist(sensorFileName, 'file') && ~exist(gpsFileName, 'file')
        break;
    end
    index = index + 1;
end
```

```
% Escribir encabezados
fidSensor = fopen(sensorFileName, 'w');
fprintf(fidSensor, 'Fecha,Hora,Minuto,Segundo,AM/PM,Temperatura (°C),Presión (hPa),Altitud (m),Humedad (%%)\n');
fclose(fidSensor);

fidGPS = fopen(gpsFileName, 'w');
fprintf(fidGPS, 'Fecha,Hora,Minuto,Segundo,AM/PM,Latitud,Longitud\n');
fclose(fidGPS);

% Buffers para almacenar los datos
dataBuffer = [];
timeBuffer = [];
locationBuffer = [];

% Variables de control
stopFlag = false;
pauseFlag = false;

% Crear botones en la figura
uicontrol('Style', 'pushbutton', 'String', 'Stop', ...
    'Position', [20, 0, 100, 30], 'Callback', @(src, event) setStopFlag());
uicontrol('Style', 'pushbutton', 'String', 'Pause', ...
    'Position', [250, 0, 100, 30], 'Callback', @(src, event) setPauseFlag());
uicontrol('Style', 'pushbutton', 'String', 'Continue', ...
    'Position', [450, 0, 100, 30], 'Callback', @(src, event) setContinueFlag());

try
    while
        ~stopFlag if
        ~pauseFlag
            % Leer línea de datos del puerto serial
            dataLine = fgetl(s);
```

```
% Separar los datos CSV
data = sscanf(dataLine, '%f,%f,%f,%f,%f,%f');

if length(data) == 6
    dataInt = round(data(1:4));
    latitude = data(5);
    longitude = data(6);

    elapsedTime = seconds(datetime('now') - startTime);
    dt = datetime('now');

    fecha = datestr(dt, 'dd-mm-yyyy');
    hora = str2double(datestr(dt, 'hh'));
    minuto = str2double(datestr(dt, 'MM'));
    segundo = str2double(datestr(dt, 'ss'));
    ampm = datestr(dt, 'AM');

    dataBuffer = [dataBuffer; dataInt'];
    timeBuffer = [timeBuffer; elapsedTime];
    locationBuffer = [locationBuffer; latitude, longitude];

    % Guardar datos del sensor
    fidSensor = fopen(sensorFileName, 'a');
    fprintf(fidSensor, '%s,%02d,%02d,%02d,%s,%d,%d,%d,%d\n', ...
        fecha, hora, minuto, segundo, ampm, dataInt(1), dataInt(2), dataInt(3), dataInt(4));
    fclose(fidSensor);

    % Guardar datos GPS
    fidGPS = fopen(gpsFileName, 'a');
    fprintf(fidGPS, '%s,%02d,%02d,%02d,%s,%.6f,%.6f\n', ...
        fecha, hora, minuto, segundo, ampm, latitude, longitude);
    fclose(fidGPS);
```

```
            % Actualizar gráficos
            for i = 1:4
                subplot(2, 2, i);
                set(h(i), 'XData', timeBuffer, 'YData', dataBuffer(:, i));
            end

            drawnow limitrate;

            fprintf('Ubicación: Latitud %.6f, Longitud %.6f\n', latitude, longitude);
        end
    else
        pause(0.1);
    end
end

disp('Lectura de datos finalizada');

catch ME
    disp('Se produjo un error: ');
    disp(ME.message);
end

% Cerrar el puerto serial
try
    fclose(s)
    ;
    delete(s)
    ; clear s;
    disp('Puerto serial cerrado');
catch ME
    disp('Error al cerrar el puerto
    serial:'); disp(ME.message);
end
```

```
% Funciones de control
function setStopFlag()
    assignin('base', 'stopFlag', true);
end

function setPauseFlag()
    assignin('base', 'pauseFlag', true);
end

function setContinueFlag()
    assignin('base', 'pauseFlag', false);
end
```

Codigo del cansat “Heisenberg”

```
#include <SPI.h>
#include <RF24.h>
#include <Adafruit_BME280.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>
#include <PWMServo.h>

#define SEALEVELPRESSURE_HPA (1013.25)

// Pines
#define GPS_RX 4
#define GPS_TX 3
const int buzzerPin = 5;

// Módulos
RF24 radio(8, 9);
Adafruit_BME280 bme;
```

```
SoftwareSerial gpsSerial(GPS_RX, GPS_TX);
TinyGPSPlus gps;
PWMServo myServo;

// RF24
const byte address[6] = "OCHO8";

// Coordenadas
struct Coordinates {
    float latitude;
    float
    longitude;
};

// Datos del sensor
struct SensorData {
    float
    temperature;
    float pressure;
    float altitude;
    float humidity;
    Coordinates coords;
};

// Variables del buzzer
bool alarmaActiva = false;
unsigned long lastBuzzerToggle = 0;
bool buzzerState = false;

// Variables de GPS float
lastValidLat = 0.0; float
lastValidLon = 0.0;
unsigned long lastGpsCheck = 0;
```

```
const unsigned long gpsInterval = 1000; // 1 segundo
```

```
void setup() {
  Serial.begin(9600);
  gpsSerial.begin(9600);

  if (!bme.begin()) {
    Serial.println("Error al inicializar el BME280");
    while (1);
  }

  if (!radio.begin()) {
    Serial.println("Error al inicializar el nRF24L01");
    while (1);
  }

  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(4);
  radio.openWritingPipe(address);
  radio.stopListening();

  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);

  // Configuración del servo
  myServo.attach(10); //Asociar el servo al pin 10
  myServo.write(0); // Posición inicial en 0 grados
}

void loop() {
  SensorData data;

  // Leer sensores BME280
```

```
data.temperature = bme.readTemperature();
data.pressure = bme.readPressure() / 100.0F;
data.altitude = bme.readAltitude(SEALEVELPRESSURE_HPA);
data.humidity = bme.readHumidity();

// Lectura de datos del GPS cada 1 segundo
unsigned long currentMillis = millis();
if (currentMillis - lastGpsCheck >= gpsInterval) {
   lastGpsCheck = currentMillis;

   // Procesar datos entrantes del GPS durante 1 segundo
   unsigned long gpsReadStart = millis();
   while (millis() - gpsReadStart < 1000) {
      while (gpsSerial.available()) {
         gps.encode(gpsSerial.read());
      }
   }

   // Validar y guardar coordenadas
   if (gps.location.isValid() && gps.location.isUpdated()) {
      lastValidLat = gps.location.lat();
      lastValidLon = gps.location.lng();
      Serial.println(" GPS actualizado correctamente.");
   } else {
      Serial.println("GPS no actualizado, se mantienen últimas coordenadas válidas.");
   }
}

// Usar las últimas coordenadas válidas
data.coords.latitude = lastValidLat;
data.coords.longitude = lastValidLon;
```

```
// Enviar datos por nRF24L01
if (radio.write(&data, sizeof(SensorData))) {
   Serial.println(" Datos enviados con éxito");
} else {
   Serial.println(" Fallo en el envío de datos");
}

// Mostrar datos por serial
Serial.print("Temp: "); Serial.print(data.temperature); Serial.print(" °C | ");
Serial.print("Pres: "); Serial.print(data.pressure); Serial.print(" hPa | ");
Serial.print("Alt: "); Serial.print(data.altitude); Serial.print(" m | ");
Serial.print("Hum: "); Serial.print(data.humidity); Serial.print(" % | ");
Serial.print("Lat: "); Serial.print(data.coords.latitude, 6); Serial.print(" | ");
Serial.print("Lon: "); Serial.println(data.coords.longitude, 6);

// Alarma por altitud
if (data.altitude >= 1700) {
   alarmaActiva = true;
}

if (alarmaActiva) {
   if (millis() - lastBuzzerToggle >= 500) {
      lastBuzzerToggle = millis();
      buzzerState = !buzzerState;
      digitalWrite(buzzerPin, buzzerState);
   }
} else {
   digitalWrite(buzzerPin, LOW);
}

// Control del servo según la altitud
if (data.altitude >= 1700.0) {
```

```
    myServo.write(180); // Mover a 180 grados
  }
}
```

Conclusión final

El proyecto Cansat representó una experiencia educativa enriquecedora, llena de desafíos y aprendizajes significativos. Este dispositivo, que simula un satélite en un formato similar al de una lata de 355 ml, nos permitió llevar a cabo una misión autónoma de recolección y transmisión de datos ambientales como temperatura, humedad, presión atmosférica y altitud sobre el nivel del mar.

Durante el desarrollo del proyecto, se integraron conocimientos de electrónica, programación, redes y diseño, lo que nos brindó una valiosa experiencia práctica en el ámbito de la tecnología espacial. El proceso completo, desde el diseño y la construcción hasta la operación del sistema, nos permitió fortalecer habilidades técnicas, capacidades de gestión y análisis de datos.

Al incluir de un sistema de paracaídas, añadió un componente adicional de ingeniería y experimentación, elevando el nivel de complejidad del proyecto. El Cansat no solo impulsó el aprendizaje teórico, sino que también nos preparó para enfrentar retos reales en entornos profesionales, al fomentar el trabajo en equipo, la integración de múltiples disciplinas y la resolución efectiva de problemas prácticos.